DBMS Lab Sheet 1

Lab Sheet 1: DDL Commands - CREATE, ALTER and DROP

Q1. Create a database named CollegeDB.

```
CREATE DATABASE IF NOT EXISTS CollegeDB;
USE CollegeDB;
SHOW DATABASES;
```

USE selects CollegeDB as the active database for every statement that follows.

Q2. Create a Student table with columns Student_ID, Name, Age, Course and City.

```
CREATE TABLE Student (
    Student_ID INT,
    Name       VARCHAR(50),
    Age        INT,
    Course     VARCHAR(30),
    City       VARCHAR(30)
);
DESCRIBE Student;
```

Q3. Create an Employee table with Emp_ID, Emp_Name, Department and Salary.

```
CREATE TABLE Employee (
    Emp_ID     INT,
    Emp_Name   VARCHAR(50),
    Department VARCHAR(30),
    Salary     DECIMAL(10,2)
);
DESCRIBE Employee;
```

Q4. Create a Department table with Dept_ID, Dept_Name and Location.

```
CREATE TABLE Department (
    Dept_ID   INT,
    Dept_Name VARCHAR(50),
    Location  VARCHAR(50)
);
DESCRIBE Department;
```

Q5. Create a Product table with Product_ID, Product_Name, Price and Quantity.

```
CREATE TABLE Product (
    Product_ID   INT,
    Product_Name VARCHAR(50),
    Price        DECIMAL(10,2),
    Quantity     INT
);
DESCRIBE Product;
```

Q6. Create a Customer table with Customer_ID, Customer_Name, Email and Phone.

```
CREATE TABLE Customer (
    Customer_ID   INT,
    Customer_Name VARCHAR(50),
    Email         VARCHAR(80),
    Phone         VARCHAR(15)
);
DESCRIBE Customer;
```

Phone is stored as VARCHAR because phone numbers are identifiers, not quantities to calculate with.

Q7. Create a Course table with Course_ID, Course_Name, Duration and Fees.

```
CREATE TABLE Course (
    Course_ID   INT,
    Course_Name VARCHAR(50),
    Duration    VARCHAR(20),
    Fees        DECIMAL(10,2)
);
DESCRIBE Course;
```

Q8. Create a Faculty table with Faculty_ID, Faculty_Name, Subject and Salary.

```
CREATE TABLE Faculty (
    Faculty_ID   INT,
    Faculty_Name VARCHAR(50),
    Subject      VARCHAR(50),
    Salary       DECIMAL(10,2)
);
DESCRIBE Faculty;
```

Q9. Create a Library table with Book_ID, Book_Name, Author and Price.

```
CREATE TABLE Library (
    Book_ID   INT,
    Book_Name VARCHAR(100),
    Author    VARCHAR(50),
    Price     DECIMAL(8,2)
);
DESCRIBE Library;
```

Q10. Create a Department table with Dept_ID as the Primary Key.

```
DROP TABLE IF EXISTS Department;      -- the table already exists from Q4
CREATE TABLE Department (
    Dept_ID   INT PRIMARY KEY,
    Dept_Name VARCHAR(50),
    Location  VARCHAR(50)
);
DESCRIBE Department;
```

Tables from Q2 to Q5 already exist, so Q10 to Q13 drop and recreate them with constraints.

Q11. Create a Student table with Student_ID as Primary Key and Name as NOT NULL.

```
DROP TABLE IF EXISTS Student;
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name       VARCHAR(50) NOT NULL,
    Age        INT,
    Course     VARCHAR(30),
    City       VARCHAR(30)
);
-- Test: this row is rejected (Error 1048: Column 'Name' cannot be null)
INSERT INTO Student VALUES (1, NULL, 20, 'BCA', 'Meerut');
```

Q12. Create an Employee table with a UNIQUE constraint on Email.

```
DROP TABLE IF EXISTS Employee;
CREATE TABLE Employee (
    Emp_ID     INT PRIMARY KEY,
    Emp_Name   VARCHAR(50) NOT NULL,
    Department VARCHAR(30),
    Email      VARCHAR(80) UNIQUE
);
INSERT INTO Employee VALUES (1, 'Sunita Rao', 'IT', 'sunita@college.edu');
-- Test: rejected, same e-mail again (Error 1062: Duplicate entry)
INSERT INTO Employee VALUES (2, 'Ritu Saxon', 'IT', 'sunita@college.edu');
```

Salary is left out on purpose so that it can be added with ALTER TABLE in Q19.

Q13. Create a Product table with a CHECK constraint that Price > 0.

```
DROP TABLE IF EXISTS Product;
CREATE TABLE Product (
    Product_ID   INT PRIMARY KEY,
    Product_Name VARCHAR(50) NOT NULL,
    Price        DECIMAL(10,2) CHECK (Price > 0),
    Quantity     INT
);
INSERT INTO Product VALUES (1, 'Laptop', 55000.00, 10);   -- accepted
-- Test: rejected (Error 3819: Check constraint is violated)
INSERT INTO Product VALUES (2, 'Pen', -5.00, 100);
```

CHECK constraints are enforced from MySQL 8.0.16 onwards; older versions parse but ignore them.

Q14. Add a new column Email to the Student table using ALTER TABLE.

```
ALTER TABLE Student ADD COLUMN Email VARCHAR(80);
DESCRIBE Student;
```

Q15. Add a Phone column to the Student table.

```
ALTER TABLE Student ADD COLUMN Phone VARCHAR(15);
DESCRIBE Student;
```

Q16. Modify the datatype/size of the Name column in the Student table.

```
ALTER TABLE Student MODIFY COLUMN Name VARCHAR(100) NOT NULL;
DESCRIBE Student;
```

MODIFY replaces the full column definition, so NOT NULL is repeated to keep that rule.

Q17. Rename the Student table to Student_Details.

```
RENAME TABLE Student TO Student_Details;
-- equivalent form
-- ALTER TABLE Student RENAME TO Student_Details;
SHOW TABLES;
```

From here until Q21 the table is called Student_Details.

Q18. Rename the column City to Address.

```
ALTER TABLE Student_Details RENAME COLUMN City TO Address;
-- for MySQL versions before 8.0:
-- ALTER TABLE Student_Details CHANGE City Address VARCHAR(30);
DESCRIBE Student_Details;
```

Q19. Add a Salary column to the Employee table.

```
ALTER TABLE Employee ADD COLUMN Salary DECIMAL(10,2) DEFAULT 0.00;
DESCRIBE Employee;
```

Q20. Drop the Phone column from the Student table.

```
ALTER TABLE Student_Details DROP COLUMN Phone;
DESCRIBE Student_Details;
```

Q21. Drop the Email column from the Student table.

```
ALTER TABLE Student_Details DROP COLUMN Email;
DESCRIBE Student_Details;
```

Q22. Remove all records from a table using TRUNCATE.

```
INSERT INTO Library VALUES
    (1, 'Database System Concepts', 'Silberschatz', 650.00),
    (2, 'Let Us C', 'Yashavant Kanetkar', 420.00);
SELECT * FROM Library;          -- 2 rows
TRUNCATE TABLE Library;
SELECT * FROM Library;          -- Empty set: rows gone, structure kept
```

TRUNCATE is a DDL command: it removes every row at once, cannot take a WHERE clause, cannot be rolled back and resets any AUTO_INCREMENT counter. DELETE FROM is the row-by-row DML alternative.

Q23. Drop the Course table from the database.

```
DROP TABLE Course;
SHOW TABLES;
```

Use DROP TABLE IF EXISTS Course; to avoid an error when the table is already gone.

Q24. Drop the Customer table from the database.

```
DROP TABLE IF EXISTS Customer;
SHOW TABLES;
```

Q25. Create a complete Student table with Primary Key, NOT NULL, UNIQUE and CHECK constraints, and demonstrate suitable ALTER and DROP operations.

```
CREATE TABLE Student (
    Student_ID INT          PRIMARY KEY,
    Name       VARCHAR(50)  NOT NULL,
    Age        INT,
    Course     VARCHAR(30)  NOT NULL,
    City       VARCHAR(30)  DEFAULT 'Meerut',
    Email      VARCHAR(80)  UNIQUE,
    CONSTRAINT chk_age CHECK (Age BETWEEN 16 AND 60)
);

-- Valid row
INSERT INTO Student VALUES (1,'Aarav Sharma',20,'BCA','Meerut','aarav@mail.com');
-- Rejected: duplicate primary key (Error 1062)
INSERT INTO Student VALUES (1,'Priya Singh',21,'BBA','Delhi','priya@mail.com');
-- Rejected: NOT NULL on Name (Error 1048)
INSERT INTO Student VALUES (2,NULL,21,'BBA','Delhi','p2@mail.com');
-- Rejected: UNIQUE on Email (Error 1062)
INSERT INTO Student VALUES (3,'Rohan Verma',19,'B.Sc','Noida','aarav@mail.com');
-- Rejected: CHECK on Age (Error 3819)
INSERT INTO Student VALUES (4,'Sneha Gupta',10,'MCA','Noida','s@mail.com');

-- ALTER operations
ALTER TABLE Student ADD COLUMN Phone CHAR(10);
ALTER TABLE Student MODIFY COLUMN Name VARCHAR(100) NOT NULL;
ALTER TABLE Student RENAME COLUMN City TO Address;

-- DROP operations
ALTER TABLE Student DROP COLUMN Phone;
ALTER TABLE Student DROP CHECK chk_age;
ALTER TABLE Student DROP INDEX Email;     -- removes the UNIQUE constraint
DESCRIBE Student;
DROP TABLE Student;
```

Each rejected INSERT proves one constraint is active; the ALTER and DROP statements then change and finally remove the table.